```matlab
% Προσομοίωση: Phase-coded FMCW Laser Headlamp for ISCAI

clear; clc; close all;

%% ΒΑΣΙΚΕΣ ΠΑΡΑΜΕΤΡΟΙ ΣΥΣΤΗΜΑΤΟΣ
c = 3e8;                % Ταχύτητα φωτός (m/s)
fc = 193.4e12;          % Συχνότητα λέιζερ (193.4 THz)
lambda = c/fc;
B = 10e9;               % Bandwidth 10 GHz
Tc = 10e-6;             % Chirp duration 10 us
mu = B/Tc;              % Chirp slope
Rb = 1e9;               % Data rate 1 Gbps
M = 128;                % Αριθμός chirps (slow time)
Fs = 4*B;               % Συχνότητα δειγματοληψίας

% Για την προσομοίωση RDM θα χρησιμοποιήσουμε ισοδύναμο baseband μοντέλο
N = 512;                % Samples per chirp
dt = Tc/N;
t_fast = (0:N-1)*dt;

disp('Ξεκινά η προσομοίωση των 7 βημάτων...');

%% 1 & 2. PC-FMCW, DPSK MODULATION & NOISE
num_symbols = 10; % Σύμβολα ανά chirp (scaled)
data_bits = randi([0 1], M, num_symbols);
phase_code = zeros(M, N);

for m = 1:M
    sym_phase = cumsum(data_bits(m,:)*pi); % DPSK (0, pi)

    % Υπολογίζουμε λίγα παραπάνω δείγματα με το ceil() για να μην ξεμείνουμε
    temp_phase = repelem(mod(sym_phase, 2*pi), ceil(N/num_symbols));

    % Κρατάμε ακριβώς τα N πρώτα δείγματα (δηλαδή 512)
    phase_code(m,:) = temp_phase(1:N);
end

%% 3. CRLB, FIM & FIGURE 2 (Range-Doppler Maps)
targets_A = [20, 10;   % [Range(m), Velocity(m/s)]
             40, 10;
             100, -20];

targets_B = [30, 10;
             31, 10;
             100, -20];

rdm_A = generate_rdm(targets_A, M, N, Tc, lambda, c, mu, phase_code);
rdm_B = generate_rdm(targets_B, M, N, Tc, lambda, c, mu, phase_code);

figure('Name', 'Figure 2: Range-Doppler Maps', 'Position', [100, 500, 800, 350]);
subplot(1,2,1); plot_rdm(rdm_A, 'Fig 2(a) Well-separated targets', M, N);
subplot(1,2,2); plot_rdm(rdm_B, 'Fig 2(b) Closely spaced targets', M, N);

disp('Βήματα 1, 2, 3, 4, 7 (RDM & Sensing) ολοκληρώθηκαν.');
```

```
%% 4. ADB (Adaptive Driving Beam) - FIGURE 3 (ΥΨΗΛΗΣ ΑΚΡΙΒΕΙΑΣ)
car_w = 1.8;       % Πλάτος στόχου (m)
y_HL = 0.65;       % Θέση Αριστερού Προβολέα
y_HR = -0.65;      % Θέση Δεξιού Προβολέα

% -- Σενάριο 3(a): Oncoming Vehicle --
t_a = linspace(0, 9, 100);
d_a = 150 - 14.5 * t_a;
y_target_a = -25 + 3.3 * t_a;

LL_left_bound = atand((y_target_a + car_w/2 - y_HL) ./ d_a);
RL_left_bound = atand((y_target_a + car_w/2 - y_HR) ./ d_a);
LL_right_bound = atand((y_target_a - car_w/2 - y_HL) ./ d_a);
RL_right_bound = atand((y_target_a - car_w/2 - y_HR) ./ d_a);

figure('Name', 'Figure 3: ADB Shadow Angle Adjustments', 'Position', [100, 100, 900, 400]);
subplot(1, 2, 1);
plot(t_a, RL_left_bound, '--', 'Color', '#7E2F8E', 'LineWidth', 1.5); hold on;
plot(t_a, LL_left_bound, '-', 'Color', '#7E2F8E', 'LineWidth', 1.5);
plot(t_a, RL_right_bound, '--', 'Color', '#D95319', 'LineWidth', 1.5);
plot(t_a, LL_right_bound, '-', 'Color', '#D95319', 'LineWidth', 1.5);
yline(15, 'k--'); yline(-15, 'k--');
xlim([0 9]); ylim([-20 20]);
xlabel('Time (s)'); ylabel('Shadow Angle (\circ)');
title('Fig 3(a) Oncoming vehicle');
grid on;

% -- Σενάριο 3(b): Multiple Preceding Vehicles --
t_b = linspace(0, 2, 100);
d_b1 = 30 - 2 * t_b;
y_tar1 = -6.4;
d_b2 = 30 - 11.5 * t_b;
y_tar2 = -1.5;

T1_LL_L = atand((y_tar1 + car_w/2 - y_HL) ./ d_b1);
T1_RL_L = atand((y_tar1 + car_w/2 - y_HR) ./ d_b1);
T1_LL_R = atand((y_tar1 - car_w/2 - y_HL) ./ d_b1);
T1_RL_R = atand((y_tar1 - car_w/2 - y_HR) ./ d_b1);

T2_LL_L = atand((y_tar2 + car_w/2 - y_HL) ./ d_b2);
T2_RL_L = atand((y_tar2 + car_w/2 - y_HR) ./ d_b2);
T2_LL_R = atand((y_tar2 - car_w/2 - y_HL) ./ d_b2);
T2_RL_R = atand((y_tar2 - car_w/2 - y_HR) ./ d_b2);

subplot(1, 2, 2);
plot(t_b, T1_RL_L, '--', 'Color', '#77AC30', 'LineWidth', 1.5); hold on;
plot(t_b, T1_LL_L, '-', 'Color', '#77AC30', 'LineWidth', 1.5);
plot(t_b, T1_RL_R, '--', 'Color', '#4DBEEE', 'LineWidth', 1.5);
plot(t_b, T1_LL_R, '-', 'Color', '#4DBEEE', 'LineWidth', 1.5);

plot(t_b, T2_RL_L, '--', 'Color', '#7E2F8E', 'LineWidth', 1.5);
plot(t_b, T2_LL_L, '-', 'Color', '#7E2F8E', 'LineWidth', 1.5);
```

```
plot(t_b, T2_RL_R, '--', 'Color', '#D95319', 'LineWidth', 1.5);
plot(t_b, T2_LL_R, '-', 'Color', '#D95319', 'LineWidth', 1.5);
yline(15, 'k--'); yline(-15, 'k--');
xlim([0 2]); ylim([-20 20]);
xlabel('Time (s)'); ylabel('Shadow Angle (\circ)');
title('Fig 3(b) Multiple preceding vehicles');
grid on;

disp('Βήμα 5 (ADB) ολοκληρώθηκε.');

%% ΣΕΝΑΡΙΟ 1: Δύο διασταυρούμενες γραμμικές τροχιές (Subplots a, b, c, d)
% Παραγωγή Ground Truth
x_true = 10:2:90;
y_true1 = x_true;                % Γραμμή 1: y = x
y_true2 = -x_true + 100;         % Γραμμή 2: y = -x + 100

% Προσθήκη Gaussian Noise
noisy_y1 = y_true1 + 1.5 * randn(size(x_true));
noisy_y2 = y_true2 + 1.5 * randn(size(x_true));

% Προσθήκη τυχαίου Clutter (Θόρυβος υποβάθρου)
num_clutter = 80;
clutter_x = rand(1, num_clutter) * 100;
clutter_y = rand(1, num_clutter) * 100;

all_x1 = [x_true, x_true, clutter_x];
all_y1 = [noisy_y1, noisy_y2, clutter_y];

% Υπολογισμός Hough Transform (Μόνο για το xy projection)
theta = 0:1:179;
rho = zeros(length(all_x1), length(theta));
for i = 1:length(all_x1)
    % Εξίσωση Rho = x*cos(theta) + y*sin(theta)
    rho(i,:) = all_x1(i)*cosd(theta) + all_y1(i)*sind(theta);
end

% Δημιουργία Accumulator (2D Histogram)
theta_flat = repmat(theta, length(all_x1), 1);
theta_flat = theta_flat(:);
rho_flat = rho(:);

theta_edges = linspace(0, 180, 181);
rho_edges = linspace(-150, 150, 301);
H_space = histcounts2(theta_flat, rho_flat, theta_edges, rho_edges)';
theta_centers = theta_edges(1:end-1) + 0.5;
rho_centers = rho_edges(1:end-1) + (rho_edges(2)-rho_edges(1))/2;

% Εξομάλυνση (Clustering / Mean filter 3x3 όπως αναφέρει το paper)
H_smooth = conv2(H_space, ones(3)/9, 'same');

% ----------------- Subplot (a) -----------------
subplot(2,3,1);
scatter(all_x1, all_y1, 10, [0.7 0.7 0.7], 'filled'); hold on; % Detected Points (Clutter)
```

```
plot(x_true, y_true1, 'k-.', 'LineWidth', 1.5);                % True Track
plot(x_true, y_true2, 'k-.', 'LineWidth', 1.5);
plot(x_true, noisy_y1, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 1); % Detected Track
plot(x_true, noisy_y2, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);
xlim([0 100]); ylim([-40 100]);
xlabel('x'); ylabel('y');
legend('Detected Points', 'True Track', 'Detected Track', 'Location', 'south');
title('(a)'); grid on;

% ----------------- Subplot (b) -----------------
subplot(2,3,2);
imagesc(theta_centers, rho_centers, H_space);
axis xy; colormap(gca, 'hot');
xlabel('Theta'); ylabel('Rho');
title('(b)');

% ----------------- Subplot (c) -----------------
subplot(2,3,3);
imagesc(theta_centers, rho_centers, H_smooth);
axis xy; colormap(gca, 'hot');
xlabel('Theta'); ylabel('Rho');
title('(c)');

% ----------------- Subplot (d) -----------------
subplot(2,3,4);
imagesc(theta_centers, rho_centers, H_smooth);
axis xy; colormap(gca, 'hot'); hold on;
% Εύρεση και σχεδιασμός Peaks (Προσεγγιστικά βάσει των γωνιών)
plot(135, 70, 'gx', 'MarkerSize', 10, 'LineWidth', 2); % Peak 1 (135 μοίρες)
plot(45, 0, 'gx', 'MarkerSize', 10, 'LineWidth', 2);   % Peak 2 (45 μοίρες)
xlabel('Theta'); ylabel('Rho');
legend('Detected Peaks', 'Location', 'northeast', 'Color', 'w');
title('(d)');


%% ΣΕΝΑΡΙΟ 2: Μία γραμμική & Μία καμπύλη τροχιά (Subplots e, f)
% Παραγωγή Ground Truth
x2_true = 10:2:90;
y2_lin = x2_true;                                  % Linear track
y2_nonlin = -0.015 * (x2_true - 60).^2 + 45;       % Non-linear track (Παραβολή)

% Προσθήκη πυκνού Clutter
num_clutter2 = 200;
clutter2_x = rand(1, num_clutter2) * 100;
clutter2_y = rand(1, num_clutter2) * 100;

all_x2 = [x2_true, x2_true, clutter2_x];
all_y2 = [y2_lin + randn(size(x2_true)), y2_nonlin + randn(size(x2_true)), clutter2_y];

% ----------------- Subplot (e) -----------------
subplot(2,3,5);
% Χρήση RGB triplets αντί για Hex strings για απόλυτη συμβατότητα
```

```
scatter(x2_true, y2_lin, 15, [0.3010, 0.7450, 0.9330], 'filled'); hold on;     % Original Points 1
scatter(x2_true, y2_nonlin, 15, [0.8500, 0.3250, 0.0980], 'filled');           % Original Points 2
scatter(clutter2_x, clutter2_y, 5, [0.8, 0.8, 0.8], 'filled');                 % Noise Points
xlim([0 100]); ylim([0 100]);
xlabel('x'); ylabel('y');
legend('Original Points 1', 'Original Points 2', 'Noise Points', 'Location', 'southeast');
title('(e)'); grid on;

% ----------------- Subplot (f) -----------------
subplot(2,3,6);
scatter(clutter2_x, clutter2_y, 5, [0.8 0.8 0.8], 'filled'); hold on; % Noise Points
plot(x2_true, y2_nonlin, 'b-', 'LineWidth', 2);                        % Detected Track 2 (Non-linear)
plot(x2_true, y2_lin, 'r-', 'LineWidth', 2);                           % Detected Track 1 (Linear)
xlim([0 100]); ylim([0 100]);
xlabel('X'); ylabel('Y');
legend('Noise Points', 'Detected Track 1', 'Detected Track 2', 'Location', 'southeast');
title('(f)'); grid on;

%% --- ΒΟΗΘΗΤΙΚΕΣ ΣΥΝΑΡΤΗΣΕΙΣ ---

function rdm = generate_rdm(targets, M, N, Tc, lambda, c, mu, phase_code)
    beat_signal = zeros(M, N);
    t_fast = linspace(0, Tc, N);

    for m = 1:M
        s_if_sum = zeros(1, N);
        for p = 1:size(targets,1)
            R = targets(p,1);
            v = targets(p,2);
            tau = 2*R/c;
            fd = 2*v/lambda;

            fb = mu * tau;
            s_target = exp(1j * 2*pi * (fb*t_fast + fd*m*Tc)) .* exp(1j * phase_code(m,:));
            s_if_sum = s_if_sum + s_target;
        end

        % ΒΗΜΑ 3: GDF (Ακύρωση phase coding)
        s_if_sum = s_if_sum .* exp(-1j * phase_code(m,:));

        noise = (randn(1,N) + 1j*randn(1,N)) * 0.5;
        beat_signal(m,:) = s_if_sum + noise;
    end

    % ---  ΑΣΦΑΛΕΣ WINDOWING ΓΙΑ ΜΑΤΡΙΧ ΔΙΑΣΤΑΣΕΙΣ ---
```

```
    window_2d = hamming(M) * hamming(N)';
    rdm = fftshift(fft2(beat_signal .* window_2d), 1);
    rdm = 10*log10(abs(rdm).^2);
end

function plot_rdm(rdm, title_str, M, N)
    range_axis = linspace(0, 120, N);
    vel_axis = linspace(-25, 25, M);

    imagesc(range_axis, vel_axis, rdm);
    axis xy; colormap(jet);
    xlabel('Range (m)'); ylabel('Velocity (m/s)');
    title(title_str);

    % Χρησιμοποιούμε caxis αντί για clim για συμβατότητα με παλαιότερες εκδόσεις MATLAB
    caxis([max(rdm(:))-30, max(rdm(:))]);

    xlim([0 120]);
end
```